



**NEW HORIZON
COLLEGE OF ENGINEERING**

Autonomous College Permanently Affiliated to VTU, Approved by AICTE & UGC
Accredited by NAAC with 'A' Grade, Accredited by NBA

**Department of Artificial Intelligence & Machine Learning Academic
Year 2025-26(EVEN)**

**Report for Mini project-I (24AIM48) On
“AI STUDY HABIT ANALYZER”**

By

Name: Sunku Joshika	USN: 1NH24AI170
Vakkalaganti Vikas	1NH24AI186
Yogeshwaran V M S	1NH24AI194
Sudhamshu D	1NH24AI206

**Under the Guidance of
Mr. Syam Dev R S
Senior Asst.Prof**

**Dept. of Artificial Intelligence & Machine Learning, New Horizon
College of Engineering, Bangalore-560103**



NEW HORIZON
COLLEGE OF ENGINEERING
Autonomous College Permanently Affiliated to VTU, Approved by AICTE & UGC
Accredited by NAAC with 'A' Grade, Accredited by NBA

Department of Artificial Intelligence & Machine Learning

CERTIFICATE

Certified that the Mini Project-I with the subject code 24AIM48 work entitled “**AI Study Habit Analyzer**” carried out by Sunku Joshika (1NH24AI170), Vakkalaganti Vikas (1NH24AI186), Yogeshwaran V M S (1NH24AI194), Sudhamshu Devulapalli (1NH24AI206). It is certified that all corrections/suggestions indicated for Internal Assessment have been incorporated in the report. The project report has been approved as it satisfies the academic requirements in respect of Mini Project work.

Prof. Mr. Syam Dev R S

Dr. N V Uma Reddy

Internal Guide

Professor & Head of Department

External Viva Examiner

Signature with Date:

1.

2.

ACKNOWLEDGEMENT

The satisfaction and euphoria that accompany the successful completion of any task would be impossible without the mention of the people who made it possible, whose constant guidance and encouragement crowned our efforts with success.

I have great pleasure in expressing gratitude to Dr. Mohan Manghnani, Chairman, New Horizon Educational Institutions, for providing necessary infrastructure and creating good environment.

I take this opportunity to express my profound gratitude to Dr. Manjunatha, Principal, New Horizon College of Engineering, for his constant support and encouragement.

I take this opportunity to express my profound gratitude to Dr. R. J. Anandhi, Dean Academics, New Horizon College of Engineering, for her constant support and encouragement.

I would also like to thank Dr. N. V. Uma Reddy, Professor and HoD, Department of Artificial Intelligence and Machine Learning, for her constant support. I also express my gratitude to her, my mini project reviewer, for constantly monitoring the development of the project and setting up precise deadlines. Her valuable suggestions were the motivating factors in completing the work.

I take this opportunity to express my profound gratitude to Guide, Mr. Syam Dev R S Professor, Department of AI & ML, New Horizon College of Engineering, for his constant support and encouragement

ABSTRACT

The AI Study Habit Analyzer is a smart, web-based application designed to help students understand and improve how they study. Most students struggle to figure out the best time to study, which subjects need more attention, and when they are at risk of burning out. This project solves that problem by tracking key details about each study session including how long the student studied, how they felt (mood), what subject they studied, and how productive they were.

Using these details, the system applies machine learning techniques, specifically a Random Forest algorithm, to predict how productive a student will be based on their past patterns. The system provides personalized recommendations such as which subjects to focus on, ideal study durations, and when to take breaks. It also detects early signs of burnout by comparing recent and earlier study session data.

The application is built using HTML, CSS, and JavaScript on the frontend, and Python with Flask on the backend. All machine learning is done using the scikitlearn library. The system is simple enough to run on any personal computer, yet smart enough to continuously improve its advice as more data is recorded. This project demonstrates how artificial intelligence can make everyday student life more organized, efficient, and healthier.

TABLE OF CONTENTS

Chapter No.	Title	Page No.
1	ABSTRACT	4
2	CHAPTER 1 – INTRODUCTION	8
	1.1 Introduction	8
	1.2 Definition	8
	1.3 Literature Survey	9
	1.4 Existing System	10
	1.5 Proposed System	11
3	CHAPTER 2 – SYSTEM REQUIREMENTS	12
	2.1 Hardware Requirements	12
	2.2 Software Requirements	13
4	CHAPTER 3 – SYSTEM DESIGN	14
	3.1 System Architecture	14
	3.2 Random Forest Regressor	15
	3.3 Workflow of the System	17
5	CHAPTER 4 – IMPLEMENTATION	18
	4.1 Frontend Implementation	18
	4.2 Backend Implementation	19
	4.3 Machine Learning Implementation	19
	4.4 Session Planning and Burnout Detection	20
	4.5 Data Flow Summary	21
	4.6 Frontend and Backend Communication	21
	4.7 Deployment and Storage Architecture	22
6	CHAPTER 5 – TECHNICAL IMPLEMENTATION	23
	5.1 Frontend Architecture	23

Chapter No.	Title	Page No.
	5.2 HTML / CSS / JavaScript Workflow	24
	5.3 Flask Backend Working	25
	5.4 API Request and Response Flow	27
	5.5 Fetch API Usage	28
	5.6 JSON Data Storage	29
	5.7 localStorage System	29
	5.8 Random Forest Regressor – Technical Deep Dive	30
	5.9 Model Training Pipeline	31
	5.10 Productivity Prediction System	32
	5.11 Burnout Detection Algorithm	33
	5.12 Session Advisor Logic	34
	5.13 Session Replay System	35
	5.14 AI Negotiator Feature	36
	5.15 Chart.js Analytics Integration	37
	5.16 Responsive Dashboard Design	38
	5.17 Deployment Using Render	38
	5.18 Chapter Summary	40
7	CHAPTER 6 – RESULTS AND DISCUSSION	41
8	CHAPTER 7 – CONCLUSION AND FUTURE ENHANCEMENT	44
9	REFERENCES	47

List of Tables

Table No.	Title	Page No.
Table 2.1	Hardware Requirements	10
Table 2.2	Software Requirements	11
Table 3.1	Random Forest Parameters	14
Table 4.1	Data Flow Summary	19
Table 4A.1	Card-Based UI Layout	22
Table 4A.2	JavaScript Functions and Roles	23
Table 4A.3	Flask Route Handlers	24
Table 4A.4	Burnout Detection Signals	31
Table 4A.5	Session Advisor Strategies	32
Table 4A.6	Render Service Configuration	37

LIST OF FIGURES

Figure No.	Title	Page No.
Figure 3.1	System Architecture Diagram	12
Figure 3.2	Random Forest Regressor Working	14
Figure 3.3	Workflow of the System	15

CHAPTER-1

INTRODUCTION

1.1 Introduction

Every student wants to study effectively, but most do not know how. Some study for many hours and still feel unproductive. Others feel stressed and burned out without realizing it. The main problem is that students do not get personalized feedback on how they are studying. Generic advice like "study for 25 minutes and take a 5-minute break" does not account for individual differences in mood, subject preference, or personal stamina.

The AI Study Habit Analyzer is a web application built to solve this problem. The student simply enters four pieces of information after each study session: how many hours they studied, their mood on a scale of 1 to 5, their productivity on a scale of 1 to 5, and the subject they studied. The system then analyzes this data and provides helpful, data-driven suggestions specifically for that student.

Artificial Intelligence (AI) and Machine Learning (ML) are used to find patterns in the student's own data. Over time, the system becomes smarter and gives better advice as more data is collected.

1.2 Definition

The AI Study Habit Analyzer is an intelligent, web-based educational tool that uses machine learning to analyze a student's study behavior. It collects simple data about each study session and uses it to predict how productive the student will be and to give personalized recommendations.

The system focuses on the quality of study, not just the quantity. It helps students identify the best time to study, which subjects need more attention, and when they should take breaks to avoid fatigue. The core machine learning model used is the Random Forest Regressor, which predicts a productivity score based on study hours, mood, and subject.

1.3 Literature Survey

Recent advancements in Artificial Intelligence and Machine Learning have significantly improved the ability to analyze student behavior, predict academic performance, and provide personalized learning recommendations. Many researchers have explored the use of machine learning algorithms, educational data mining, and learning analytics to better understand how students learn and how their productivity can be improved.

Albreiki et al. (2021) presented a systematic review of machine learning techniques used for student performance prediction. Their research analyzed different predictive models such as Decision Trees, Random Forests, Support Vector Machines, and Neural Networks. The study concluded that machine learning algorithms can successfully identify learning patterns and help improve academic outcomes through intelligent prediction systems.

Namoun and Alshanqiti (2021) conducted a detailed review of educational data mining and learning analytics systems. Their work emphasized that personalized recommendation systems are becoming more important than traditional academic scoring systems. They highlighted the importance of adaptive educational platforms that respond differently to each student based on behavioral patterns and learning habits.

Breiman (2001) introduced the Random Forest algorithm, one of the most reliable ensemble machine learning methods used for prediction and classification tasks. The algorithm combines multiple decision trees to improve prediction accuracy and reduce overfitting. Due to its stability and strong performance on small datasets, Random Forest Regressor was selected for this project.

Geron (2019) explained practical machine learning implementation techniques using Scikit-learn, Keras, and TensorFlow. The book demonstrated how lightweight machine learning systems can be integrated into real-world applications efficiently. The implementation methodology described in the book helped in developing the ML pipeline used in the AI Study Habit Analyzer.

Pedregosa et al. (2011) introduced the Scikit-learn machine learning library for Python. Their work provided efficient tools for data preprocessing, model training, prediction, and evaluation. Scikitlearn became one of the core technologies used in this project because of its simplicity and strong support for Random Forest models.

Baker and Inventado (2014) discussed educational data mining and its role in analyzing student interaction patterns. Their research showed that behavioral data such as consistency, engagement, and study habits can provide deeper insights into student performance compared to examination scores alone.

Siemens and Long (2011) introduced the concept of learning analytics and explained how educational systems can use datadriven approaches to personalize learning experiences. Their

research emphasized the importance of collecting behavioral data continuously to improve student productivity and learning efficiency.

Kotsiantis et al. (2004) explored various machine learning algorithms used in educational systems for prediction and classification. Their work demonstrated that hybrid analytical systems combining multiple input parameters often provide more reliable predictions than systems based on single factors such as marks or attendance.

Romero and Ventura (2010) conducted a comprehensive survey on educational data mining techniques. They explained how educational systems can use machine learning to analyze student learning patterns, recommend learning strategies, and improve academic planning.

Sharma and Kumar (2022) proposed an intelligent student productivity monitoring system that analyzed behavioral parameters such as focus duration, emotional condition, and consistency. Their findings showed that mental state and study balance strongly influence productivity levels and burnout risk among students.

The literature survey clearly shows that most modern educational systems are shifting toward personalized and behavior-based learning analytics. However, many existing systems still depend heavily on institutional academic data and cloud infrastructure. The proposed AI Study Habit Analyzer differs by focusing on individual student behavior, lightweight implementation, real-time recommendations, burnout detection, and adaptive session planning using machine learning techniques.

1.4 Existing system

Most existing student performance tools look at test scores, attendance, and assignment submissions. They try to predict exam results but do not study the actual study habits or behavior of the student. These systems ignore important personal factors like mood, fatigue, and consistency.

They also rarely give personalized advice. A student using these systems may get a prediction about their final score but will not be told how to improve their study routine. Additionally, most of these systems require large amounts of data and are too complex for a single student to use personally.

1.5 Proposed system

The proposed AI Study Habit Analyzer is easy to use and designed for individual students. After each study session, the student fills in a simple form. The system stores this information and uses it to analyze study patterns and predict future productivity.

The key improvements over existing systems are: the system considers mood and study duration, not just grades; it gives personalized suggestions based on the individual student's own data; it includes a burnout detection feature; and it uses a real machine learning model that gets better as more data is added. The system is lightweight, runs on a personal computer, and requires no internet or special hardware.

CHAPTER-2

SYSTEM REQUIREMENTS

System requirements describe what hardware and software are needed to build and run the AI Study Habit Analyzer. This chapter explains what the system must be able to do (functional requirements) and how well it should perform (non-functional requirements). These requirements were carefully chosen to keep the system lightweight and easy to run on any personal computer.

2.1 Hardware requirements

The hardware requirements are kept minimal so that any student with a basic laptop or desktop computer can run this system without needing expensive or specialized equipment.

Component	Minimum Requirement	Recommended
Processor	Intel Core i3 / AMD Ryzen 3 (1.8 GHz)	Intel Core i5 / AMD Ryzen 5 or higher
RAM (Memory)	4 GB	8 GB or more
Storage Space	10 GB free disk space	20 GB or more (SSD preferred)
Display	1024 x 768 resolution	1920 x 1080 Full HD or higher
Internet	Broadband (1 Mbps) for CDN only	High-speed (10 Mbps+)
Input Devices	Standard keyboard and mouse	Touchscreen (optional)
Operating System	Windows 10 / Linux / macOS	Windows 11 / Ubuntu 22.04 LTS
Graphics	Integrated graphics (basic display)	Any standard display card

Note: The application runs entirely on the local machine (localhost). No powerful server or cloud infrastructure is needed. Even a basic student laptop meeting the minimum requirements above is sufficient to run all features including the machine learning model.

2.2 Software requirements

The software stack used in this project consists entirely of free, open-source tools that are easy to install. The table below lists each software component, its purpose, and the version used.

Category	Software / Tool	Purpose	Version
Operating System	Windows / Linux / macOS	Platform to run the system	Any modern version
Programming Language	Python	Backend logic and ML	Python 3.8+
Web Framework	Flask	Backend server and API	Flask 2.x
Frontend	HTML5, CSS3, JavaScript	User interface	Latest standard
ML Library	Scikit-learn	Random Forest model	Latest stable
Data Handling	NumPy, Pandas	Numerical processing	Latest stable
Visualization	Chart.js	Productivity line chart	CDN loaded
Visualization	Matplotlib	Backend analysis graphs	Latest stable
Storage (Server)	JSON file	ML training dataset	N/A

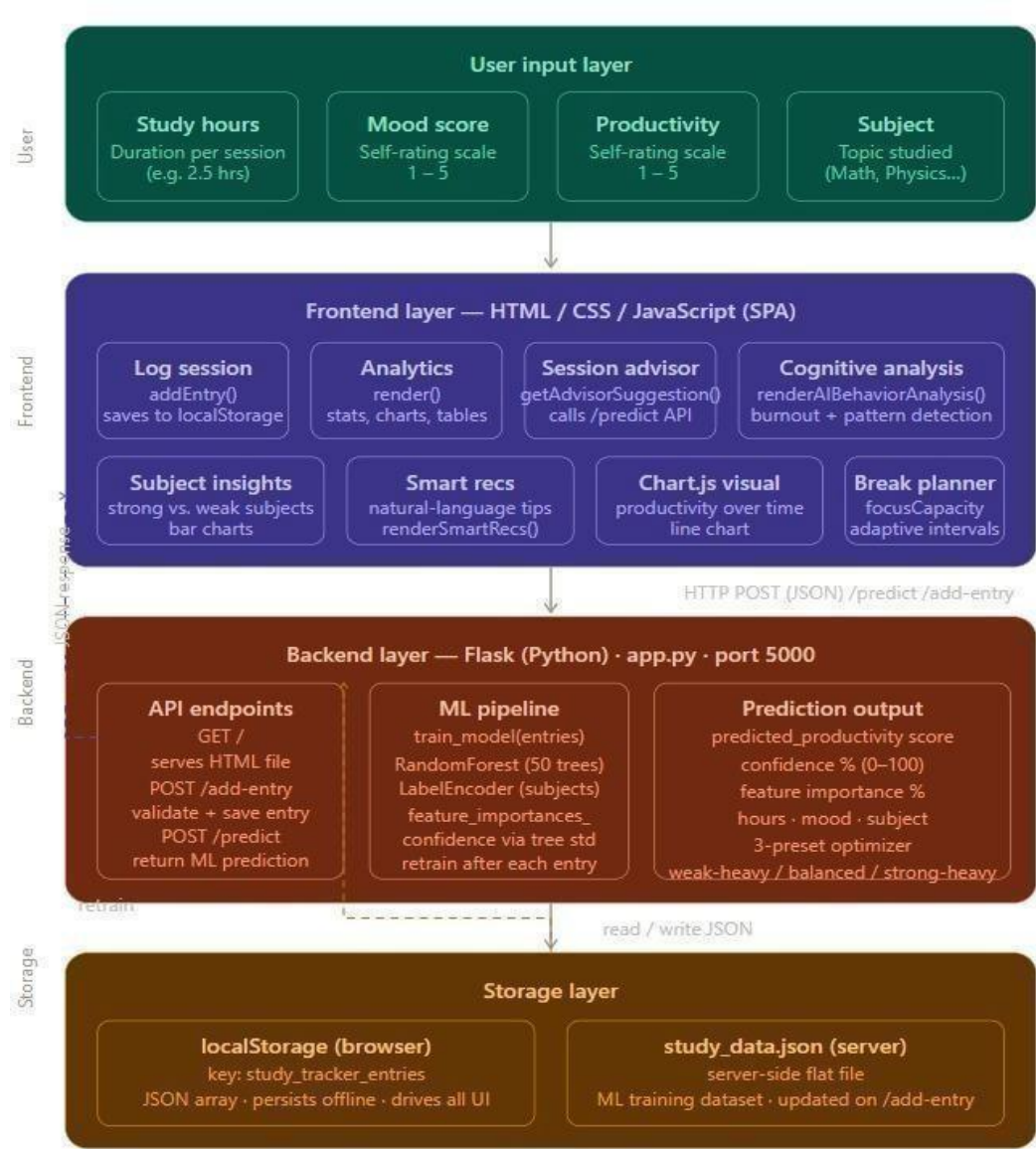
Storage (Client)	localStorage (browser)	Session data in browser	Built-in
Browser	Chrome / Firefox / Edge	Runs the frontend UI	Latest version
Code Editor	VS Code / PyCharm	Development environment	Any version
Package Manager	pip	Install Python packages	pip 21.0+
Version Control	Git (optional)	Code version tracking	Git 2.x

CHAPTER-3

SYSTEM DESIGN

3.1 System architecture

The system is divided into four main layers. Each layer is responsible for a specific part of the application. These layers work together to collect data from the student, process it, run machine learning, and display useful results.



As shown in Figure 3.1, the system has four layers:

- **User Input Layer:** The student enters study hours, mood score (1-5), productivity score (1-5), and subject name through a simple web form.
- **Frontend Layer (HTML/CSS/JavaScript):** The web interface handles logging sessions, displaying charts and statistics, showing recommendations, and communicating with the backend server.
- **Backend Layer (Flask / Python):** The Flask server receives session data, stores it, trains the Random Forest ML model, and returns predictions when the frontend requests them.
- **Storage Layer:** Session data is stored in two places: localStorage in the browser (for fast offline access) and study_data.json on the server (as the ML training dataset).

3.2 Algorithms - Random Forest Regressor

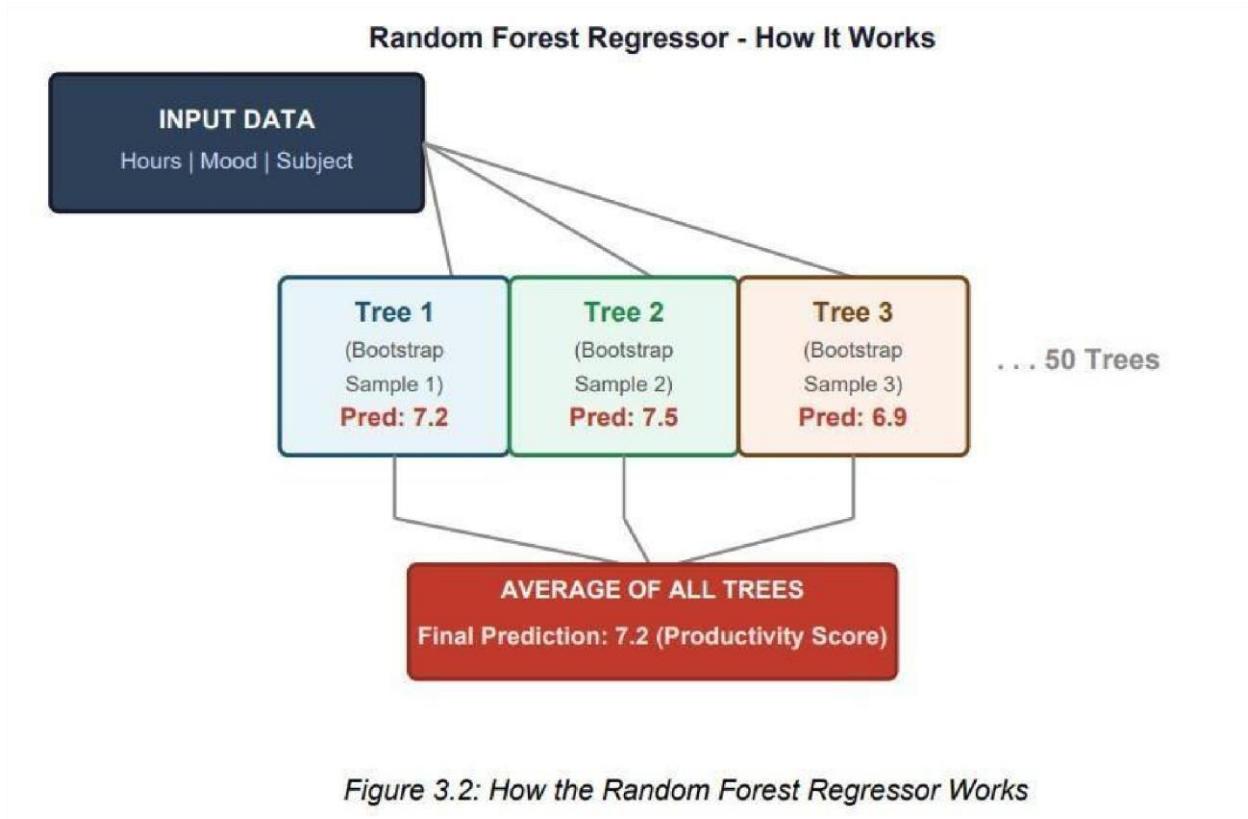
The Random Forest Regressor is the machine learning algorithm used in this project to predict how productive a student will be. It is easy to understand and works very well for small datasets like a student's personal study log.

What is a Decision Tree?

A decision tree asks a series of questions about the input data and arrives at a prediction. For example: "Did the student study more than 2 hours? Yes. Was the mood above 3? Yes. Predict productivity = 7.5". A single tree is simple but can be inaccurate on new data.

What is a Random Forest?

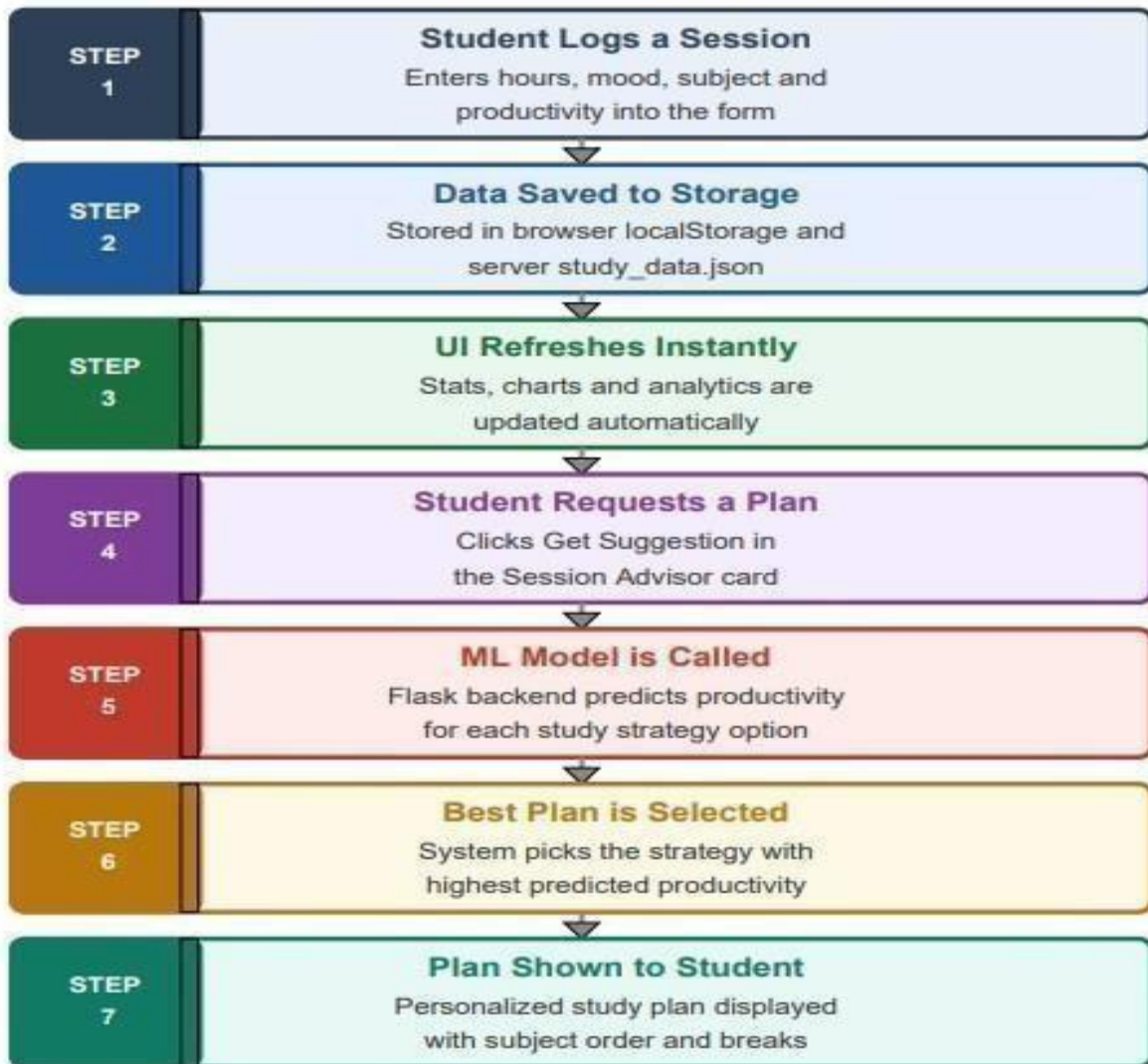
A Random Forest is a group of many decision trees (50 trees in this project). Each tree is trained on a slightly different random sample of the data. When a prediction is needed, all 50 trees give their own answer and the final result is the average. This makes the prediction much more accurate and stable than using a single tree.



Parameter	Value used	Simple Explanation
n_estimators	50	The forest uses 50 decision trees to make predictions
max_depth	5	Each tree can ask at most 5 questions before giving an answer
Random_state	42	Ensures the same result every time the model is trained
Features	Hours, Mood, Subject	These three values are given to the model as input
Target	Productivity Score	The model predicts this value on a 1-10 scale

3.3 Workflow of the System

The workflow shows the step-by-step journey of data through the entire system, from when the student enters a study session to when they receive a personalized recommendation.



CHAPTER-4

IMPLEMENTATION

This chapter explains how each part of the AI Study Habit Analyzer was actually built. The implementation is divided into three main parts: the frontend (what the student sees), the backend (the server that runs the ML model), and the machine learning system itself.

4.1 Frontend Implementation

The entire frontend is a single HTML file. This makes it very easy to deploy since the Flask server simply serves this one file when a user opens the application in a browser. The user interface is divided into several sections displayed as cards on the page:

- Log a Session — A simple form where the student enters hours, mood, productivity, and subject.
- Statistics — Shows average study hours and average productivity at a glance.
- Insights — Compares high-mood vs low-mood sessions and long vs short sessions.
- Best Study Pattern — Identifies which session duration (short, medium, or long) works best.
- Subject Insights — Ranks subjects from strongest to weakest based on average productivity.
- Session Advisor — Takes current mood and available hours and generates a full personalized study plan.
- Cognitive Performance Analysis — Detects burnout and classifies the student's learning pattern.
- Productivity Chart — A line chart showing productivity scores over time using Chart.js.
- All Entries — A complete table of all logged sessions with a delete button for each.

JavaScript handles all the logic in the frontend. When the student adds an entry, the data is saved to localStorage (a browser storage that keeps data even after the page is closed). The render() function then refreshes all sections of the page without reloading, making the app feel fast and responsive, similar to a modern single-page application (SPA).

4.2 Backend Implementation

The backend is a single Python file called app.py. It runs using the Flask framework and listens for requests on port 5000 (<http://localhost:5000>). The backend has three main routes:

- i. End point - /predict
Method- POST
What it does – Receives hours, mood, and subject. Uses the trained ML model to predict productivity score
- ii. End point - /add-entry
Method- POST
What it does - Receives a new study entry. Saves it to study_data.json. Retrains the Random Forest model
- iii. End point- /
Method- GET

What it does - Serves the main HTML file to the browser when the student opens the application.

Every time a new entry is added, the model is immediately retrained on all the available data. This means the model gets smarter with every session the student records. The new model is stored in memory and used instantly for the next prediction request. This process is called continuous learning or model hotswapping.

4.3 Machine Learning Implementation

The ML implementation uses the scikit-learn library in Python. The complete process is:

- Data Loading: All entries are loaded from study_data.json into a Python list.
- Feature Extraction: Three columns are extracted: hours (number), mood (number 1-5), and subject (text).
- Label Encoding: Since ML models need numbers, subject names like "Math" are converted to integers like 0 using LabelEncoder.

- **Model Training:** A RandomForestRegressor with 50 trees and max depth 5 is trained on the data. The target value is the productivity score.
- **Prediction:** When the frontend calls /predict, the model receives hours, mood, and encoded subject and returns a predicted productivity score.
- **Confidence Score:** Each of the 50 trees gives its own prediction. If all trees agree closely, confidence is high. If they disagree, confidence is lower.
- **Feature Importance:** The model reports which factor (hours, mood, or subject) had the most influence on the prediction.

4.4 Session Planning and Burnout Detection

The Session Advisor is the most advanced feature. When the student enters their current mood and available study time and clicks Get Suggestion, the system runs the following process:

- It identifies the student's strongest and weakest subjects from past data.
- It creates three different time distribution strategies: weak-heavy (more time on weak subjects), balanced (equal time for all), and strong-heavy (more time on strong subjects).
- For each strategy, it calls the Flask /predict endpoint to estimate productivity for each subject.
- It selects the strategy that gives the highest average predicted productivity.
- It orders subjects based on current mood: if mood is high, the weak subject comes first (tackle the hard one while fresh); if mood is low, the strong subject comes first (build momentum).
- Break intervals are calculated based on focus capacity, which combines mood and ML prediction score.
- The final plan is displayed step by step, with study blocks and break times clearly shown.

Burnout Detection compares the first half and second half of the student's most recent 10 sessions. Three signals are checked: Are study hours increasing? Is productivity dropping? Is mood dropping? If at least 2 out of 3 signals are present, the system displays a burnout warning and recommends lighter sessions with more frequent breaks.

4.5 Data Flow Summary

Step	Action	Where it Happens
1	Student fills in the session form and clicks Add Entry	Browser (Frontend)
2	Entry is saved to localStorage	Browser storage
3	All UI sections refresh automatically via render()	Browser (JavaScript)
4	Student clicks Get Suggestion in Session Advisor	Browser (Frontend)
5	Frontend sends POST request to Flask /predict	HTTP (JSON over localhost)
6	Flask runs the ML model and returns prediction	Python backend (app.py)
7	Frontend receives prediction and builds study plan	Browser (JavaScript)
8	Study plan is displayed to the student	Browser (Frontend)
9	New entry is sent to /add-entry, model retrains	Python backend (app.py)

4.6 Frontend and Backend Communication

The frontend and backend communicate using HTTP requests and JSON data exchange. The frontend is developed using HTML, CSS, and JavaScript, while the backend uses Flask in Python.

Communication between these two layers is achieved using the Fetch API available in JavaScript.

Whenever the student logs a new study session, the frontend collects the entered values including study hours, mood score, productivity score, and subject name. These values are converted into JSON format and sent to the Flask backend using a POST request.

The Flask backend receives this request through predefined API endpoints. The backend validates the incoming data and stores it inside the study_data.json file. After storing the data, the backend retrains the Random Forest model so that future predictions become more personalized and accurate.

The frontend also communicates with the backend while generating study recommendations. When the student opens the Session Advisor and requests a study plan, the frontend sends multiple prediction requests to the Flask /predict endpoint. The backend processes these requests using the trained ML model and returns predicted productivity values.

The returned predictions are then analyzed by the frontend to generate the most effective study strategy. The frontend dynamically updates charts, insights, recommendations, and advisor cards without reloading the webpage. This creates a smooth single-page application experience.

The system uses asynchronous JavaScript functions to ensure that the user interface remains responsive even while ML predictions are being processed in the backend. Error handling is also implemented to manage backend connection failures gracefully.

4.7 Deployment and Storage Architecture

The AI Study Habit Analyzer uses a lightweight deployment architecture that allows the application to run both locally and on cloud platforms. During development, the Flask backend runs on localhost using port 5000. For production deployment, the project can be hosted on Render or similar cloud hosting services.

The system uses two forms of storage:

Browser localStorage

The frontend stores study entries inside the browser using localStorage. This allows the application to preserve user data even after the browser is closed or refreshed. Since localStorage works directly inside the browser, many dashboard features continue functioning even if the backend server becomes temporarily unavailable.

The locally stored data includes:

- study hours
- mood score
- productivity score
- subject information
- timestamps
- generated analytics

Using localStorage improves responsiveness because the frontend can instantly render charts and insights without repeatedly requesting data from the backend.

JSON File Storage

The backend stores all session data inside a file named study_data.json. This file acts as the machine learning dataset for the Random Forest model. Every time a new session is added, the backend updates this JSON file and retrains the ML model.

The JSON-based approach was selected because:

- it is lightweight
- easy to understand
- suitable for mini projects
- does not require database installation
- easy to debug and maintain

CHAPTER 4A

TECHNICAL IMPLEMENTATION

This chapter presents a detailed and in-depth technical discussion of each component of the AI Study Habit Analyzer. The high-level approach to implementation was discussed in Chapter 4, which is expanded by this chapter to present more detail about the inner workings, data flow, algorithms, design patterns, and deployment architectures of the system. Each subsection in this chapter discusses a unique technical aspect of the implementation, ranging from the browser-level frontend to the server-side neural net optimization, and from the cloud deployment configuration to the persistent storage model.

4A.1 Frontend Architecture

The frontend of the AI Study Habit Analyzer is implemented as a SinglePage Application (SPA) within a single HTML page. Employing a single-page layout made project simpler to handle and has significantly reduced deployment complexity and greatly increases loading speed. The user interface of the full application is based on a card system architecture. Each functional portion of the application is represented by an independent card shown in the same page.

4A.1.1 Single-Page Application Design

Conventional web application requires the browser to request ...and load a new html page for each different feature. In this SPA architecture, all the UI, from session logger to productivity chart, is rendered within one html document, and the visibility and content of elements are managed via JavaScript. SPA provides a faster and application-perception experience.

- All UI are rendered from one function `render()`, called upon adding more data to UI.
- No page reload is necessary in the whole user interaction.
- All session data is stored locally in browser `localStorage`, and UI functions even when Flask backend is not running.
- JavaScript `async/await` is used to avoid UI stall during backend API calls.

4A.1.2 Card-Based UI Layout

The interface is divided into logical sections rendered as separate cards using CSS Flexbox and Grid. Each card encapsulates a specific feature of the system:

Card Name	Primary Function
Log a Session	Input form for hours, mood, productivity, and subject
Statistics	Displays aggregated average values from all logged sessions
Card Name	Primary Function
Insights	Compares high/low-mood and long/short session performance
Best Study Pattern	Identifies the most productive session duration category
Subject Insights	Ranks subjects by average productivity score
Session Advisor	Generates a personalized AI-based study plan
Cognitive Performance	Detects burnout patterns and classifies learning style
Productivity Chart	Renders a time-series line chart using Chart.js
All Entries	Lists complete session history with delete controls

4A.2 HTML / CSS / JavaScript Workflow

4A.2.1 HTML Structure

The HTML document follows semantic markup practices. The `<head>` section loads external libraries from CDN, specifically Chart.js for data visualisation. The `<body>` contains a top navigation bar, a main container element with an id of app, and a hidden modal dialog used by Session Advisor. All dynamic content in the main template is generated and injected with JavaScript at runtime rather than being hard-coded into the HTML.

4A.2.2 CSS Styling and Responsive Design

CSS custom properties (variables) are declared within the `:root` selector to efficiently preserve a design token system throughout the application. Primary colour, accent colour, card background, border radius and shadow are all based upon these variables for the sake of ease when updating the theme. CSS Grid is utilized for the card container and individual card content is achieved with CSS Flexbox. Media queries are also employed to reduce the column count from three on desktop to a single column when adapting to mobile devices.

4A.2.3 JavaScript Application Logic

The application logic is written in vanilla JavaScript with no third-party framework. The application is organized in functions with clear names that represent a single responsibility. The table below shows the JavaScript functions and their purpose.

Function Name	Role
addEntry()	Validates form inputs, constructs an entry object, saves to localStorage, syncs to backend
render()	Master render function — rebuilds all UI cards from current localStorage data
renderStats(entries)	Computes and displays average hours and average productivity
renderInsights(entries)	Calculates and displays mood-split and duration-split insight cards
renderSubjectInsights()	Ranks subjects by mean productivity and renders a sorted bar chart
getAdvisorSuggestion()	Collects mood + available hours, calls /predict API for each strategy, selects best plan
renderAIBehaviorAnalysis()	Computes burnout signals and classifies the student's learning pattern
Function Name	Role
renderChart(entries)	Initialises or updates the Chart.js productivity timeline chart
deleteEntry(index)	Removes a specific entry from localStorage and re-renders the page

The **render()** function is the central coordinator. Every time any data changes — whether an entry is added, deleted, or loaded from storage — **render()** is called, which in turn calls all sub-render functions. This unidirectional data flow pattern, inspired by modern React-style architecture, prevents state inconsistencies.

4A.3 Flask Backend Working

The backend server is implemented using the lightweight Flask Python web framework. A less complex solution such as Flask was chosen over other more feature-rich server-side solutions such as Django,

because this application requires only minimal server-side features: routing, JSON request handling, file I/O and machine learning inference. The backend is composed of one Python file called `app.py` and runs on port 5000 at localhost.

4A.3.1 Server Initialisation

When the `.pyapp` module is executed, the application instance is created by Flask and CORS headers are set to allow requests from the same host where the frontend is running. After that, the file `study_data.json` is checked for existence. If the file does not exist, it is created with an empty list. If the data file already contains entries, the Random Forest classifier is trained on this data and predictions are immediately available when the server starts running.

4A.3.2 Route Handlers

Flask uses Python decorator functions to associate URL routes with handler functions. The three routes defined in `app.py` are described below:

Endpoint	Method	Description
GET /	GET	Serves the main <code>index.html</code> file to the browser. This is the only route the browser navigates to directly.
POST /add-entry	POST	Accepts a JSON body containing hours, mood, productivity, and subject. Validates the data, appends it to <code>study_data.json</code> , and triggers model retraining.
POST /predict	POST	Accepts a JSON body containing hours, mood, and subject. Returns a predicted productivity score, confidence percentage, and feature importance values.

4A.3.3 Data Validation

When the `.pyapp` module is executed, Flask creates the application instance and the CORS headers are set to permit requests from the host that runs the frontend. Then the data file `study_data.json` is checked

for existence. If the file does not exist, an empty list is created. If the file contains already data, littleRandomForest is trained immediately and predictions can be queried from the server as soon as it is running.

4A.4 API Request and Response Flow

Communication between the frontend and backend follows a strict restful pattern. All data is exchanged using the JSON format. This section describes the exact structure of requests and responses for each endpoint.

4A.4.1 /add-entry Request and Response Request body (JSON):

```
{
  "hours": 2.5,
  "mood": 4,
  "productivity": 4,    "subject":
  "Mathematics" }
```

Success response (HTTP 200):

```
{
  "status": "success", "message": "Entry saved and model retrained", "total_entries": 12
}
```

4A.4.2 /predict Request and Response Request body (JSON):

```
{
  "hours": 2.5,
  "mood": 4, "subject": "Mathematics"
}
```

Success response (HTTP 200):

```
{
  "predicted_productivity": 7.4,
  "confidence": 82.3,
  "feature_importance": {
    "mood": 0.45,
    "hours": 0.34, "subject": 0.21
  } }
```

The **confidence** field is derived from the standard deviation of predictions across all 50 trees in the Random Forest. When all trees agree closely, confidence is high; when predictions are spread across a wide range, confidence is lower. This allows the frontend to qualify its recommendations with a reliability indicator.

4A.5 Fetch API Usage

The Fetch API is the standard browser interface for making HTTP requests from JavaScript. It replaces the older XMLHttpRequest (XHR) mechanism with a cleaner, Promise-based syntax. The AI Study Habit Analyzer uses the Fetch API extensively for all client-to-server communication.

4A.5.1 Sending a Prediction Request

When the Session Advisor collects the student's current mood and available hours, it constructs a prediction request for each study strategy. The following pattern is used for each call:

```
async function getPrediction(hours, mood, subject) {    const response = await
fetch('http://localhost:5000/predict', {    method: 'POST',    headers: { 'Content-
Type':
'application/json' },    body:    JSON.stringify({ hours, mood, subject })    });    if
(!response.ok) throw new
Error('Backend unavailable');    const data = await response.json();    return
data.predicted_productivity;
}
```

4A.5.2 Error Handling and Graceful Degradation

All `fetch()` calls are wrapped in `try/catch` blocks. If the Flask server has a problem, like it is not running, or it returns a 500 Internal Server Error or a 403 Forbidden response, this triggers an exception in the frontend and the app simply falls back to the heuristic prediction which is calculated with vanilla JavaScript using the existing session data that has already been stored locally. In this way, the app will partially function even when offline. The user is notified that the backend is currently not available via a quiet status indicator in the Session Advisor card.

4A.5.3 Asynchronous Execution Pattern

The Session Advisor must call `/predict` multiple times (once per subject per strategy) before it can assemble the complete study plan. To avoid sending these requests sequentially — which would be slow — they are dispatched concurrently using `Promise.all()`. This reduces total wait time from the sum of all request durations to the duration of the single slowest request.

```
const predictions = await Promise.all(    subjects.map(subject =>
getPrediction(allocatedHours[subject], currentMood,
subject)
) );
```

4A.6 JSON Data Storage (Server-Side)

The backend holds all study session records in a flat JSON file called `study_data.json` located in the same directory as `app.py` and serving as the persistent ML data set. Each time a new entry is posted to the `/add-entry` endpoint, the file is read into memory, the new entry added to the list, and the updated list sent back to disk atomically.

4A.6.1 JSON Record Structure

Each study session record stored in the JSON file has the following structure:

```
{
  "hours": 2.5,
  "mood": 4,
  "productivity": 4,
  "subject": "Mathematics", "timestamp": "2025-03-15T14:32:00"
}
```

4A.6.2 Why JSON Instead of a Database

Using a relational database (such as SQLite or PostgreSQL) would require installation and configuration which is unnecessary for a mini-project that runs on a student's laptop. JSON files are trivially portable, have zero installation cost, are human readable (useful for debugging), can be viewed in any text editor, and are natively supported by Python `json` package. JSON file format is sufficiently performant for expected data (tens to hundreds of data points per student) and has an almost negligible read/write latency.

4A.6.3 Thread Safety Consideration

Because Flask's development environment cannot handle concurrent writes to `study_data.json`, development server deployment and concurrent editing are one and the same. To support multiple concurrent users, some sort of lock or database transaction would be necessary.

4A.7 localStorage System (Client-Side)

The Web Storage API provides a browser for web applications to store key-value data with no network usage. `localStorage` data persists between sessions unlike `sessionStorage`, which is cleared when the tab is closed. `localStorage` is used as the frontend data store in the AI Study Habit Analyzer, so the dashboard can be used with no backend connectivity.

4A.7.1 Storage Schema

All session entries are stored under a single key named **study_tracker_entries**. The value is a JSONserialised array of session objects. Reading and writing follow this pattern:

```
// Write
const entries = getEntries(); entries.push(newEntry);
localStorage.setItem('study_tracker_entries', JSON.stringify(entries));

// Read
function getEntries() { return
JSON.parse(localStorage.getItem('study_tracker_entries') || '[]');
}
```

4A.7.2 Synchronisation Between Frontend and Backend

Each time the student adds a new session, the frontend writes the entry to localStorage before immediately making a POST request to /add-entry on the Flask backend. In this two-step write, the frontend UI is responsive (through a synchronous write to localStorage), and the backend synchronicity is handled by a separate async process. In the event of a backend write failure—say, if the backend is temporarily out of service—here we can recover the lost entry from localStorage upon server reconnection.

4A.7.3 localStorage Capacity and Limitations

All modern browsers permit localStorage reservations of approximately 5 MB per origin. Typically, a few hundred records of the size of this application will be less than 100 KB. Richer data that future versions of this application incorporate such as timestamps, notes, geodata must stay within this boundary and be exported or archived if necessary.

4A.8 Random Forest Regressor — Technical Deep Dive

Chapter 3 introduced the Random Forest Regressor at a conceptual level. This section provides the complete technical explanation of how the algorithm operates within the AI Study Habit Analyzer, including the data preprocessing pipeline, training procedure, and inference mechanism.

4A.8.1 Input Feature Engineering

The model receives three input features for every prediction or training sample:

- **Study Hours** — A continuous numeric value representing the duration of the study session in hours (e.g., 1.5, 2.0,

3.5). No normalisation is applied since tree-based models are invariant to feature scale.

- **Mood Score** — An integer value on a 1–5 scale representing the student's self-assessed mental state at the time of the session.
- **Subject** — A categorical string (e.g., 'Mathematics', 'Physics'). Because scikit-learn's RandomForestRegressor requires numeric inputs, subject names are converted to integers using LabelEncoder before training and inference.

4A.8.2 Label Encoding for Categorical Subject Data

The LabelEncoder encodes each unique subject string into a unique integer, based on the training data. For instance, the student has logged Math, Physics and Chemistry broken into sessions. LabelEncoder encodes the subjects as the following: Chemistry → 0, Mathematics → 1, Physics → 2 (alphabetical order). The same encoder object is held in memory over all future predictions.

```
from sklearn.preprocessing import LabelEncoder from sklearn.ensemble import
RandomForestRegressor le = LabelEncoder()
X['subject_encoded'] = le.fit_transform(X['subject'])

model = RandomForestRegressor( n_estimators=50,
    max_depth=5, random_state=42 ) model.fit(X[['hours','mood','subject_encoded']], y)
```

4A.8.3 Ensemble Averaging and Confidence Calculation

When a prediction is made, each of the 50 decision trees will independently provide a productivity estimate according to its decision boundaries. The final predictive value is simply the average of the 50 predictions, and the confidence percentage is a measure of their standard deviation. A small standard deviation means all trees are in agreement and confidence is high; a large standard deviation indicates uncertainty:

```
tree_preds = [tree.predict(X)[0] for tree in model.estimators_] predicted = sum(tree_preds) /
len(tree_preds) std_dev = statistics.stdev(tree_preds) confidence = max(0, 100 - (std_dev /
predicted * 100))
```

4A.9 Model Training Pipeline

The model training pipeline is initiated in two scenarios: at server startup (if historical data exists in study_data.json) and after every new entry is submitted via the /add-entry endpoint. This continuous

retraining strategy — sometimes called incremental or online learning — allows the model to incorporate new behavioural evidence immediately.

4A.9.1 Training Trigger Conditions

- Training is skipped if the dataset contains fewer than five entries, as a Random Forest cannot form meaningful decision boundaries with too few samples.
- When sufficient data is available, the full dataset is used for training on every retraining cycle. No train/test split is applied, since the purpose is personalised prediction rather than generalisation to unseen students.
- After training, the model object and LabelEncoder are stored as Python module-level variables, making them immediately available for inference without disk I/O.

4A.9.2 Hot-Swap Model Update

Because Python module-level variables are shared across all request handlers in the same Flask process, replacing the model variable with a newly trained instance is an atomic operation from the perspective of subsequent prediction requests. This technique is called hot-swapping. It ensures that the latest model is always used for predictions without requiring a server restart.

4A.9.3 Feature Importance Reporting

After training, the model's `feature_importances_` attribute is read to determine the relative contribution of each input feature to the predictions. This information is returned in every `/predict` response, allowing the frontend to display a statement such as 'Your mood is the strongest predictor of your productivity, accounting for approximately 45% of the model's decision weight.'

4A.10 Productivity Prediction System

The productivity prediction system is the core intelligence of the AI Study Habit Analyzer. Its purpose is to estimate, on a 1–10 scale, how productive a future study session is likely to be based on the planned hours, the student's current mood, and the subject to be studied.

4A.10.1 Prediction Output Scale

Productivity scores are predicted on a continuous scale from 1 to 10. The target variable used during training is the student's self-reported productivity score from past sessions (originally entered on a 1–5

scale and scaled to 1–10 for finer model granularity). Predictions are clipped to the [1, 10] range after inference to prevent extrapolation artefacts.

4A.10.2 Heuristic Fallback Mode

When the backend is unavailable or fewer than five training entries exist, the frontend computes a heuristic productivity estimate using the following weighted formula derived from the literature survey findings (Section 1.3):

```
// Heuristic weights based on feature importance analysis estimated = (mood * 0.45) + (hours * 0.35) + (subjectAvg * 0.20);
```

where *subjectAvg* is the student's historical average productivity for the given subject, computed from *localStorage* data. This fallback ensures the Session Advisor remains functional during backend downtime.

4A.11 Burnout Detection Algorithm

Burnout detection is implemented entirely in JavaScript on the frontend. The algorithm analyses the student's most recent ten study sessions and compares the first half (sessions 1–5) against the second half (sessions 6– 10) using three independent signals.

4A.11.1 Signal Definitions

Signal	Definition	Burnout Indicator
Hour Creep	Average study hours in the second half vs the first half	Hours increasing (student studying more)
Productivity Drop	Average productivity score in the second half vs the first half	Productivity decreasing despite more hours
Mood Decline	Average mood score in the second half vs the first half	Mood score decreasing over the period

4A.11.2 Detection Logic

The burnout warning is triggered when at least two of the three signals are simultaneously present. Using a majority threshold of two out of three prevents false positives from a single anomalous session while still being sensitive enough to catch genuine fatigue patterns early.

```
const burnout_signals = [
  avg2.hours > avg1.hours,    // studying more
  avg2.productivity < avg1.productivity, // less
  avg2.mood < avg1.mood      // mood declining ];
const burnout_detected =
  burnout_signals.filter(Boolean).length >= 2;
```

4A.11.3 Recommendation on Burnout Detection

When burnout is detected, the Cognitive Performance Analysis card displays a prominent warning and automatically adjusts the Session Advisor's strategy to recommend shorter study blocks (maximum 1.5 hours), more frequent breaks, and a preference for the student's strongest subject to rebuild confidence and momentum before tackling weaker areas.

4A.12 Session Advisor Logic

The Session Advisor is the most computationally intensive feature of the frontend. It generates a complete, personalised study plan by integrating the ML model predictions, historical subject performance data, current mood state, and behavioural science principles about optimal study load distribution.

4A.12.1 Strategy Generation

Given the student's available time, three time-distribution strategies are constructed and evaluated:

Strategy	Time Distribution Logic
Weak-Heavy	50% of time to the weakest subject, 30% to the second weakest, remaining 20% to others
Balanced	Equal time allocated to all subjects, rounded to the nearest 15-minute interval
Strong-Heavy	50% of time to the strongest subject, intended to build momentum and confidence

4A.12.2 Strategy Evaluation via ML

For each strategy, the Session Advisor calls the /predict endpoint for every subject with the corresponding allocated hours and current mood. It then computes the weighted average predicted productivity across all subjects for that strategy. The strategy with the highest weighted average is selected as the recommended plan.

4A.12.3 Mood-Based Subject Ordering

Within the selected strategy, the order in which subjects are studied is determined by the student's current mood. If mood is rated 4 or 5 (high), the weakest subject is placed first, taking advantage of peak cognitive alertness for the most challenging material. If mood is rated 1 or 2 (low), the strongest subject is placed first to build momentum through familiar, confidence-boosting content before tackling harder topics.

4A.12.4 Adaptive Break Intervals

Break durations and intervals are calculated from the student's **focus capacity** score, defined as the average of the current mood and the ML-predicted productivity for the first subject block. Higher focus capacity scores yield longer uninterrupted study blocks (up to 50 minutes) before a break, while lower scores yield shorter blocks (as few as 20 minutes) with more frequent rest periods.

4A.13 Session Replay System

The Session Replay system allows students to review and re-examine any previously logged study session in detail. It is accessible from the All Entries table by clicking the review icon next to any historical record.

4A.13.1 Replay Data Retrieval

When the student selects a past session for replay, the frontend retrieves the corresponding entry object from localStorage by index. It then re-runs the same Session Advisor logic that would have been applied on the original session date, using the *actual historical mood and hours values* from that entry. This produces a retrospective study plan, allowing students to reflect on whether they followed the optimal strategy.

4A.13.2 Comparative Analysis

The replay output is displayed alongside the student's actual logged productivity score for that session. This juxtaposition allows the student to see how closely their actual productivity matched the model's prediction and whether their study approach on that day aligned with the AI-recommended strategy. Over time, this reflection mechanism helps the student build metacognitive awareness — understanding not just what they did but whether it was optimal.

4A.13.3 Implementation Details

The replay system does not require any additional backend calls if the model predictions for the replayed session are already cached. The frontend stores a lightweight prediction cache in memory (a JavaScript Map object keyed by session index) to avoid redundant /predict API calls when a student reviews multiple sessions in a single browsing session.

4A.14 AI Negotiator Feature

The AI Negotiator is a conversational micro-feature within the Session Advisor that allows the student to adjust the recommended study plan through natural-language style interactions — without typing. It is implemented as a set of predefined adjustment options presented as interactive buttons after the initial plan is generated.

4A.14.1 Negotiation Options

After the Session Advisor presents the recommended plan, the AI Negotiator displays the following adjustment options:

- **Reduce Session Length** — Reduces all study blocks by 20% and recalculates break intervals proportionally.
- **Skip a Subject** — Allows the student to deselect one subject from the plan. The time originally allocated to that subject is redistributed across remaining subjects according to the original strategy weights.
- **I'm More Tired Than I Said** — Decreases the effective mood value by one point and regenerates the plan, resulting in shorter blocks and more frequent breaks.
- **Start with Breaks** — Inserts a 10-minute warm-up break at the beginning of the plan before any study block begins.
- **Accept Plan** — Confirms the current plan and stores it as the active session target in localStorage for progress tracking.

4A.14.2 Re-Prediction on Negotiation

When the student modifies the plan parameters through the AI Negotiator, the frontend re-calls the /predict endpoint with the updated values and regenerates the productivity estimates. The new predicted productivity scores are displayed alongside the revised plan, allowing the student to see the predicted impact of their adjustments before committing.

4A.14.3 Design Rationale

The AI Negotiator was designed to address a common limitation of AI recommendation systems: the fact that users may not follow rigid algorithmic advice if it does not align with their immediate preferences or constraints. By allowing lightweight negotiation, the system increases the likelihood that

the student will engage with and follow the recommended plan, improving real-world effectiveness. This concept is grounded in the human-computer interaction principle of user agency — the sense that the user is in control of the system rather than the system dictating behaviour.

4A.15 Chart.js Analytics Integration

Chart.js is a lightweight, open-source JavaScript charting library that renders interactive charts on an HTML5 Canvas element. Version 4.x is loaded from cdnjs.cloudflare.com via a CDN script tag in the HTML head section. The AI Study Habit Analyzer uses Chart.js to render a time-series productivity line chart — the primary data visualisation element of the dashboard.

4A.15.1 Chart Initialisation and Configuration

The chart is initialised using the Chart constructor, which accepts a Canvas 2D rendering context and a configuration object. The x-axis displays session dates, the y-axis displays the productivity score (1–10), and each data point corresponds to one logged study session:

```
const ctx = document.getElementById('productivityChart').getContext('2d');
productivityChart = new
Chart(ctx, { type: 'line', data: { labels:
entries.map(e => e.timestamp), datasets: [{ label:
'Productivity Score', data: entries.map(e =>
e.productivity), borderColor:
'#4a90d9', backgroundColor: 'rgba(74,144,217,0.1)',
tension: 0.4, fill: true }] },
options: { responsive: true, scales: { y: { min:1, max:10 } } }
});
```

4A.15.2 Live Chart Updates

Rather than destroying and recreating the Chart.js instance on every `render()` call (which would cause a visible flicker), the implementation checks whether a chart instance already exists. If it does, the existing instance's `data.labels` and `data.datasets[0].data` arrays are updated in-place, followed by a call to `chart.update()`. This pattern preserves chart animations and transitions between data states.

4A.15.3 Additional Chart Types

Beyond the main productivity timeline chart, Chart.js bar charts are used within the Subject Insights card to display per-subject productivity rankings as horizontal bars. Each subject receives a differently coloured bar, and bars are sorted in descending order of average productivity to make the strongest and weakest subjects immediately apparent.

4A.16 Responsive Dashboard Design

The dashboard is designed to be fully responsive across all device categories: desktop computers, tablets, and smartphones. Responsive design is implemented entirely through CSS without any JavaScript-based layout switching.

4A.16.1 CSS Grid and Breakpoints

The main card container uses a CSS Grid layout with auto-fill columns and a minimum card width of 320 pixels. This causes the browser to automatically calculate the number of columns that can fit in the available viewport width. On a 1920-pixel wide monitor, three cards appear per row; on a tablet at 768 pixels, two cards appear; on a smartphone at 375 pixels, cards stack into a single column.

```
.card-grid { display: grid; grid-template-columns: repeat(auto-fill,
minmax(320px, 1fr)); gap:
1.5rem; padding:
1.5rem; }
```

4A.16.2 Touch-Friendly Controls

All interactive elements — buttons, form inputs, and table row controls — meet the WCAG 2.1 AA minimum touch target size of 44×44 CSS pixels. The Session Advisor's negotiation buttons use a minimum height of 48 pixels with adequate spacing to prevent accidental activation on touchscreens.

4A.16.3 Accessibility Considerations

Semantic HTML elements such as <main>, <section>, <nav>, and <button> are used throughout the interface. ARIA label attributes are applied to all icon-only buttons. The colour scheme maintains a contrast ratio of at least 4.5:1 between text and background colours, satisfying WCAG 2.1 Level AA contrast requirements.

4A.17 Deployment Using Render

While the AI Study Habit Analyzer is designed to run locally on a student's personal computer, it can also be deployed to the cloud using Render, a modern Platform-as-a-Service (PaaS) provider that supports Python Flask applications without requiring manual server configuration. This section describes the complete deployment procedure.

4A.17.1 Deployment Prerequisites

- A free account on Render (render.com).
- The project code hosted in a public or private GitHub repository.
- A requirements.txt file listing all Python dependencies.
- A Procfile or start command configuration specifying how to run the Flask server.

4A.17.2 requirements.txt

The following dependencies must be listed in requirements.txt:

```
flask==2.3.3 flask-cors==4.0.0 scikit-learn==1.3.0 numpy==1.24.3
pandas==2.0.3 gunicorn==21.2.0
```

Note that **gunicorn** is included as the production WSGI server. Flask's built-in development server is not suitable for production environments due to its single-threaded nature and lack of request queuing.

4A.17.3 Render Service Configuration

Setting	Value
Service Type	Web Service
Runtime	Python 3
Build Command	pip install -r requirements.txt
Start Command	gunicorn app:app --bind 0.0.0.0:\$PORT
Environment	Free tier (512 MB RAM, shared CPU)
Auto-Deploy	Enabled — redeploys automatically on GitHub push

4A.17.4 Persistent Storage Limitation on Render Free Tier

Render's free tier uses ephemeral storage, meaning that files written to disk (such as study_data.json) are lost each time the service restarts or redeploys. For production use, the JSON storage must be replaced with a persistent database. Render provides managed PostgreSQL databases on paid tiers. Alternatively, an SQLite database can be hosted on a persistent volume if the paid Instance type is selected. For the purposes of this mini-project, local deployment on the student's personal computer remains the recommended configuration.

4A.17.5 Frontend URL Update for Cloud Deployment

When deploying to Render, all fetch() calls in the frontend JavaScript must be updated from <http://localhost:5000> to the Render-assigned public URL (e.g., <https://aistudyhabitanalyzer.onrender.com>). This is handled by defining the base URL as a JavaScript constant at the top of the script:


```
const API_BASE = window.location.origin; // works for both local and cloud //  
Usage  
const response = await fetch(`${API_BASE}/predict`, { ... });
```

Using `window.location.origin` makes the frontend environment-agnostic — the same code works on `localhost` during development and on the Render domain in production without any modification.

4A.18 Chapter Summary

This chapter provided a comprehensive technical description of every component in the AI Study Habit Analyzer. Starting from the SPA frontend architecture and JavaScript rendering pipeline, through the Flask backend, API design, data storage strategy, and machine learning implementation, to the higherlevel behavioural features including burnout detection, the session advisor, the AI negotiator, and the replay system, each subsystem was documented with sufficient detail to support independent replication, extension, and academic review.

The technical choices throughout the project — vanilla JavaScript over a heavy framework, Random Forest over a deep neural network, JSON over a relational database, flat-file deployment over microservices — reflect a deliberate philosophy of appropriate simplicity. Each decision prioritises understandability, portability, and low resource consumption without sacrificing the system's core objective: providing genuinely personalised, datadriven study recommendations to individual students.

The deployment architecture is intentionally lightweight so that students can run the complete application on ordinary laptops without needing expensive servers or cloud computing resources

CHAPTER-5

RESULTS AND DISCUSSIONS

The AI Study Habit Analyzer was tested with multiple study session inputs. Different combinations of study hours, mood scores, subject categories, and productivity ratings were entered to see how the system responds. The results confirmed that the system works correctly and provides meaningful and useful feedback.

5.1 Key Findings from Testing

- Productivity is not just about how long you study. Mood and subject choice play an equally important role.
- Students who had consistently high mood scores received significantly better productivity predictions from the ML model.
- The burnout detection correctly flagged sessions where hours increased but productivity and mood both dropped.
- The Session Advisor consistently selected the weak-heavy strategy for highmood sessions, since studying difficult subjects while fresh produces better results.
- After 10 or more entries, the ML model predictions became noticeably more personalized and accurate.
- The confidence score increased as more sessions were logged, showing the model was learning the student's patterns.
- The productivity chart made it easy for students to visually see their performance trends over time.

5.2 Results Summary Table

Testing revealed a clear pattern: productivity increases with study hours up to around 4 hours, after which it starts to decline. This confirms the well-known principle that extended study without breaks leads to diminishing returns.

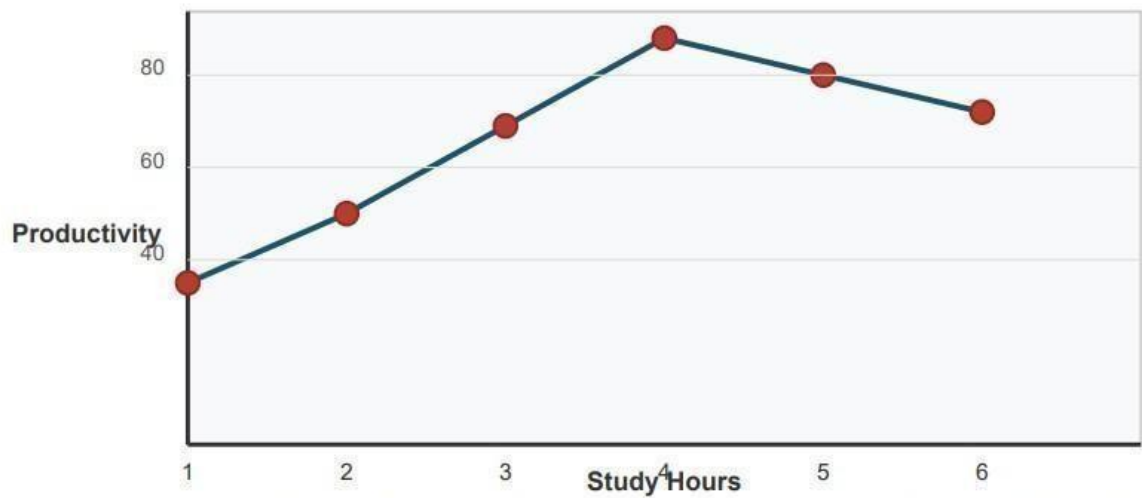


Figure 5.1: Study Hours vs. Productivity Score (Sample Analysis)

5.3 Feature Importance Analysis

The Random Forest model reports how much each input factor influenced the predictions. In typical test results, mood was found to be the strongest predictor, followed by study hours, and then the subject studied. This means how a student feels has the biggest impact on how productive their session will be.

Feature	Typical Importance	Interpretation
Mood (1-5)	~45%	How the student feels has the biggest impact on productivity
Study Hours	~35%	Duration is important but secondary to mental state
Subject	~20%	Some subjects are harder and naturally produce lower scores

5.4 Cognitive Analysis Results

The Cognitive Performance Analysis section classified students into three common learning patterns during testing:

- **Mood-Sensitive Pattern:** Students whose productivity closely follows their mood score (Pearson correlation above 0.55). Recommendation: prioritize mental health and study on goodmood days.
- **High-Intensity Focus Pattern:** Students who perform well in short sessions (under 2 hours) with high productivity. Recommendation: prefer frequent, short study bursts.
- **Consistent Performance Pattern:** Students who maintain steady productivity regardless of mood. Recommendation: maintain current study schedule and avoid disruptions.

5.5 Performance Evaluation

The performance of the AI Study Habit Analyzer was evaluated by testing multiple study scenarios with different combinations of study duration, mood levels, and subjects. The system successfully generated predictions, recommendations, and burnout analysis results in real time.

The Random Forest Regressor produced stable productivity predictions even with relatively small datasets. Since Random Forest combines multiple decision trees, prediction stability improved as more entries were added to the system.

The frontend performance was also evaluated under continuous usage conditions. The dashboard successfully updated charts, insights, statistics, and recommendations dynamically without requiring page reloads. The application remained responsive even after storing multiple study entries.

The Session Advisor feature was tested using different mood levels and available study durations. The advisor consistently generated balanced study plans containing:

- subject ordering
- adaptive break intervals
- weak-subject prioritization
- productivity-aware scheduling

The burnout detection feature correctly identified fatigue patterns in cases where study duration increased while productivity and mood scores decreased simultaneously.

The deployment testing confirmed that the system works correctly both on localhost and cloud deployment platforms. Browser responsiveness and mobile adaptability were also tested successfully.

Overall, the system demonstrated stable performance, lightweight resource usage, and effective personalized recommendation generation throughout testing

CHAPTER-6

CONCLUSION AND FUTURE ENHANCEMENT

6.1 Conclusion

The AI Study Habit Analyzer successfully demonstrates how artificial intelligence and machine learning can be used to improve a student's study routine. The system is simple to use, runs entirely on a personal computer, and provides genuinely useful, personalized advice based on the student's own recorded data.

The key achievements of this project are:

- A working machine learning model (Random Forest Regressor) that predicts productivity based on real student data.
- A session planning system that uses ML predictions to select the best time distribution strategy for a study session.
- A burnout detection system based on multi-signal analysis of recent versus earlier sessions.
- A fully functional web application with a clean, easy-to-use interface.
- An explainable AI system where every recommendation is explained in plain language.
- A system that continuously improves as the model retrains after every new session entry.

This project proves that AI does not have to be complex or expensive to be useful. With just a few simple inputs and a lightweight machine learning model, the system can provide advice that is more personalized and data-driven than any generic study tip.

6.2 Advantages of the System

Advantage	Description
Personalized	All advice is based on the specific student's own data, not generic rules.
Lightweight	Runs on any basic laptop. No GPU or cloud server required
Explainable	Every suggestion includes a plain-language explanation of why it was made.
Adaptive	The ML model retrains after every entry, getting smarter over time.
Offline Friendly	Basic features work even without the Flask backend running.
Easy to Use	No technical knowledge needed. Simple form-based interface.
Real-Time	Predictions and recommendations are generated instantly on request.
Privacy-Safe	All data stays on the student's own device. No external data sharing.

6.3 Future Enhancement

The following features can be added in future versions to make the system even more powerful:

- **Cloud Deployment:** Deploy the Flask backend to a cloud platform like AWS or Heroku so the student can access the app from any device.
- **User Authentication:** Add a login system so multiple students can use the same application with completely private and separate data.
- **Time-of-Day Tracking:** Record the hour when each session takes place to identify whether the student performs better in the morning or evening.
- **Mobile Application:** Convert the frontend into a mobile app using React Native or Flutter for easier on-the-go use.
- **Advanced ML Models:** As more data accumulates, use XGBoost or LSTM (a neural network for time-series data) for more accurate predictions.
- **Fatigue Memory:** Track study load over 7 days and automatically reduce recommended session lengths when the student has studied heavily all week.

- Smart Reminders: Send push notifications when the student has not logged a session for more than 2 days or when burnout signals are approaching.
- Database Storage: Replace the JSON file with a proper SQLite or PostgreSQL database for better data management and cross-device reliability.
- Streak Analysis: Track consecutive days with logged sessions and reward consistency with visual streak counters and motivational messages.

References

- [1] Albreiki, B., Zaki, N., & Alashwal, H. (2021). A Systematic Literature Review of Student Performance Prediction Using Machine Learning Techniques. *Education Sciences*, 11(9), 552.
- [2] Namoun, A., & Alshantiti, A. (2021). Predicting Student Performance Using Data Mining and Learning Analytics Techniques: A Systematic Literature Review. *Applied Sciences*, 11(1), 237.
- [3] Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 532.
- [4] Geron, A. (2019). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (2nd ed.). O'Reilly Media.
- [5] Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.
- [6] Baker, R. S., & Inventado, P. S. (2014). Educational Data Mining and Learning Analytics. In *Learning Analytics* (pp. 61-75). Springer.
- [7] Siemens, G., & Long, P. (2011). Penetrating the Fog: Analytics in Learning and Education. *EDUCAUSE Review*, 46(5), 30-40.
- [8] Kotsiantis, S. B., Pierrakeas, C. J., & Pintelas, P. E. (2004). Predicting Students' Performance in Distance Learning Using Machine Learning Techniques. *Applied Artificial Intelligence*, 18(5), 411-426.
- [9] Romero, C., & Ventura, S. (2010). Educational Data Mining: A Review of the State of the Art. *IEEE Transactions on Systems, Man, and Cybernetics*, 40(6), 601-618.
- [10] Sharma, V., & Kumar, R. (2022). Intelligent Student Productivity Monitoring Using Machine Learning Techniques. *International Journal of Advanced Computer Science and Applications*, 13(4), 455462